

ELO329 - Diseño y Programación Orientados a Objetos

Nociones básicas sobre C++

Agustín González

Patricio Olivares

Universidad Técnica Federico Santa María

Contenidos

- C++ frente a C y Java
- Archivos de encabezado y comentarios
- Tipos de variables y acceso a objetos
- Entrada y salida de datos
- Operadores, arreglos, vectores y `std::string`
- Control de flujo
- Punteros y paso por referencia

C++ vs C

- C++ es un lenguaje posterior a C.
- Mantiene gran parte del poder de C y agrega características que facilitan la programación.
- Es compatible hacia atrás con C en muchos aspectos.
- Permite programación procedural, orientada a objetos y genérica.
- En un mismo programa pueden mezclarse distintos paradigmas.
- Posee una biblioteca de funciones más amplia que C.
- Muchas bibliotecas existentes de C son compatibles con C++.

C++ vs Java

- C++ es dependiente de plataforma, aunque bibliotecas como Qt buscan reducir este problema.
- Soporta herencia múltiple.
- Soporta sobrecarga de operadores.
- Permite uso explícito de punteros y operaciones sobre ellos.
- Soporta paso de parámetros por valor y por referencia.
- No tiene un equivalente nativo directo a Javadoc, aunque puede usarse Doxygen.
- Cada programa C++ puede definir sus propios árboles de herencia.

Archivos de encabezado

- Permiten declarar prototipos y definir constantes usadas en el programa.
- Se incluyen con la directiva del preprocesador `#include`.
- Al usar `<...>`, el compilador busca en directorios estándar.
- Al usar `"..."`, normalmente se busca primero en rutas asociadas al proyecto.

```
#include <vector>
#include <sys/socket.h>
#include "setup.h"
```

Búsqueda de encabezados

Los directorios estándar pueden variar entre instalaciones.

Para inspeccionar dónde busca el compilador:

```
touch a.c  
gcc -v -E a.c
```

Esto muestra las rutas donde se buscan archivos incluidos mediante:

```
#include <...>  
#include "..."
```

Comentarios

C++ permite comentarios de una línea y de múltiples líneas:

```
// Comentario de una línea

/*
Comentario de múltiples líneas
*/
```

- No se permiten comentarios anidados.
- Los comentarios son extraídos por el preprocesador.
- El preprocesador no reconoce estructuras anidadas de comentarios.

Comentarios y preprocesador

Una técnica común para dejar fuera un bloque de código es usar `#if 0`:

```
#if 0
// código temporalmente excluido
// puede contener comentarios internos
#endif
```

Esto no es exactamente un comentario, sino una exclusión condicional durante el preprocesamiento.

Tipos de variables

Tipo
bool
char
short
int
long
long long
unsigned short
unsigned int
unsigned long
unsigned long long
float

Acceso de variable

En C++, las variables representan valores directamente, no referencias a valores.

En Java, esto ocurre solo con tipos simples como:

```
int, float, double, char, boolean
```

La diferencia se nota especialmente al trabajar con objetos.

Objetos en C++ y Java

En Java:

- Los nombres de objetos son referencias.
- Todo objeto se encuentra en el heap.

En C++:

- Los nombres de objetos se refieren al objeto mismo.
- Un objeto puede estar en el stack, heap o segmentos de datos.
- La creación de una variable puede invocar directamente al constructor.

Ejemplo de semántica de objetos

```
Empleado juan, pedro;  
juan = pedro;
```

Interpretación:

- Al declarar `juan` y `pedro`, se crean los objetos.
- La asignación hace que `juan` tome los atributos de `pedro`.
- Un cambio posterior en `juan` no afecta a `pedro`.

Esta semántica es una diferencia importante respecto de Java.

Salida de datos

Para usar entrada y salida estándar se incluye `iostream`.

```
#include <iostream>

int main(void) {
    std::cout << "Hello, world" << std::endl;
    return 0;
}
```

- `std::cout` permite escribir en la salida estándar.
- `std::endl` inserta un salto de línea y descarga el flujo.

Salida hacia archivos

Para trabajar con archivos se usa `fstream`.

```
#include <fstream>

std::ofstream os("output.dat");
os << "The value of pi is approx. " << 3.14159 << std::endl;
```

- `std::ofstream` permite escribir en un archivo.
- El operador `<<` también se usa para enviar datos al archivo.

Entrada de datos desde archivo

```
#include <iostream>
#include <fstream>

int main() {
    int i;
    std::ifstream fin;
    fin.open("test"); // test contiene 3 enteros

    for (int j = 0; j < 3; j++) {
        fin >> i;
        std::cout << i << std::endl;
    }

    fin.close();
}
```

Lectura línea a línea

```
#include <string>
#include <fstream>
#include <iostream>

int main() {
    std::string s;
    std::ifstream fin;
    fin.open("/etc/passwd");

    while (std::getline(fin, s))
        std::cout << s << std::endl;
}
```

`std::getline` permite leer una línea completa desde un flujo.

Operadores: precedencia

Asociatividad	Operadores
->	() [] -> .
->	! ~ ++ -- + unario - unario * & sizeof
->	/ %
->	+ -
->	<< >>
->	< <= > >=

Operadores: precedencia

Asociatividad	Operadores
->	== !=
->	& ^ `
->	&& `
<-	?:
<-	= += -= *= /= %= &= ^= `
->	,

Asignaciones, arreglos y vectores

- Toda asignación tiene un valor: el valor asignado.

```
a = b = c;
```

- ANSI C++ usa el mismo constructor de arreglos que C.
- Los arreglos de C son limitados.
- C++ incorpora vectores, análogos en uso a los `ArrayList` de Java.
- Los vectores son una forma de plantilla o `template`.

Vectores

Un vector puede crearse con espacio inicial para `n` elementos:

```
std::vector<X> a(n);
```

- `x` representa el tipo de dato.
- El acceso se realiza con corchetes:

```
a[i]
```

- A diferencia de Java, aquí aún no se usa `new`.

Vectores dinámicos

```
std::vector<double> a;  
  
a.push_back(0.3);    // a[0] = 0.3  
a.push_back(56.2);   // a[1] = 56.2
```

También se puede redimensionar:

```
a.resize(10);
```

Y consultar su tamaño:

```
for (int i = 0; i < a.size(); i++) {  
    // ...  
}
```

Strings

C++ estándar incluye una clase `std::string`.

Esta clase tiene definidos operadores como:

- Copia: `=`
- Concatenación: `+`
- Comparación: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Acceso a caracteres: `[]`

Ejemplo con strings

```
std::string s = "Hola a todos";  
  
int n = s.length();           // asigna 12  
char ch = s[0];               // primer carácter  
std::string t = s.substr(0,4); // substring desde s[0]
```

La clase `std::string` ofrece métodos útiles para manipular texto de forma más segura y cómoda que los arreglos de caracteres estilo C.

Control de flujo

C++ dispone de estructuras comunes en otros lenguajes:

```
if (condicion) {  
    // bloque 1  
} else {  
    // bloque 2  
}
```

- La parte `else` es opcional.
- Los bloques de instrucciones se delimitan con `{ }`.

Ciclos y selección

```
while (condicion) {  
    // bloque  
}
```

```
do {  
    // bloque  
} while (condicion);
```

```
for (expresion1; expresion2; expresion3) {  
    // instrucción a repetir  
}
```

switch funciona de manera análoga a Java.

Punteros: idea básica

Toda variable tiene:

- Un valor o estado.
- Una dirección de memoria asociada.

```
int miVar = 5;
```

Una variable puntero almacena la dirección en memoria de otra variable o dato.

```
miVar  
valor:      5  
dirección: 1776
```

Declaración de punteros

Para definir una variable como puntero se usa `*`:

```
int *pVar;
```

Esto indica que `pVar` puede almacenar la dirección de memoria de un entero.

Para obtener la dirección de una variable se usa `&`:

```
pVar = &miVar;
```

Punteros: referencia y dereferencia

```
std::cout << &pVar << std::endl; // dirección de pVar  
std::cout << pVar << std::endl; // contenido de pVar: dirección apuntada  
std::cout << *pVar << std::endl; // valor almacenado en la dirección apuntada
```

Operadores importantes:

- `&` : operador de referencia, obtiene la dirección de memoria.
- `*` : operador de dereferencia, accede al valor apuntado.

Diagrama conceptual de puntero

```
Variable:  miVar  
Valor:     5  
Dirección: 1776
```

```
Variable:  pVar  
Valor:     1776  
Dirección: 3571
```

Interpretación:

- `pVar` contiene la dirección de `miVar`.
- `*pVar` permite acceder al valor de `miVar`.
- `&pVar` corresponde a la dirección donde vive el puntero.

Paso por referencia

C++ agrega paso de argumentos por referencia.

- El efecto es equivalente al uso de punteros en C.
- La sintaxis es más simple.
- Permite modificar variables externas desde una función.

Swap en C usando punteros

```
void swap_en_C(int *px, int *py) {  
    int tmp = *px;  
    *px = *py;  
    *py = tmp;  
}
```

El llamado requiere pasar direcciones:

```
swap_en_C(&a, &b);
```

Swap en C++ usando referencias

```
void swap_en_Cplusplus(int &x, int &y) {  
    int tmp = x;  
    x = y;  
    y = tmp;  
}
```

El llamado es más directo:

```
swap_en_Cplusplus(a, b);
```

Las referencias permiten escribir código más simple cuando se desea modificar los argumentos originales.

Restricciones del paso por referencia

No se pueden pasar expresiones que no correspondan a variables modificables.

```
swap_en_Cplusplus(4, x);      // no es legal  
swap_en_Cplusplus(i + 2, j-1); // no es legal
```

Motivo:

- Una referencia no constante necesita asociarse a una variable real.
- No puede modificar directamente un literal o una expresión temporal.

Ideas clave

- C++ combina programación procedural, orientada a objetos y genérica.
- A diferencia de Java, los objetos pueden tener semántica de valor.
- `iostream` y `fstream` permiten trabajar con entrada y salida.
- `std::vector` y `std::string` facilitan el manejo de colecciones y texto.
- Los punteros permiten acceder explícitamente a direcciones de memoria.
- El paso por referencia simplifica operaciones que en C suelen requerir punteros.